

Generative Adversarial Networks (GANs)

1. Objective

The objective of this experiment is to design, implement, and train a **Generative Adversarial Network (GAN)** using the MNIST handwritten digits dataset. The goal is to enable the Generator to produce synthetic digit images that resemble real MNIST samples, while the Discriminator learns to distinguish real images from generated ones. The experiment evaluates:

- GAN architecture and training workflow
- Generator and Discriminator performance
- Loss behavior across epochs
- Quality of generated images
- Challenges encountered during training

2. Brief Explanation of GANs

A **Generative Adversarial Network (GAN)** consists of two neural networks trained simultaneously in a competitive setting:

Generator (G)

- Takes a random noise vector (100-dimensional)
- Upsamples it through transposed convolutions
- Produces a 28×28 grayscale image
- Objective: *fool the Discriminator into believing generated images are real*

Discriminator (D)

- Takes an image (real or generated)
- Uses convolutional layers to extract features
- Outputs a single logit (real vs. fake)
- Objective: *correctly classify real and fake images*

GAN training is adversarial:

- **G tries to minimize:** $\log(D(G(z)))$
- **D tries to minimize:** $\log(D(x)) + \log(1 - D(G(z)))$

This push-and-pull dynamic allows the Generator to gradually learn the underlying data distribution.

3. Architecture Summary

3.1 Generator Architecture

(From Colab code)

- **Input:** 100-dimensional noise vector
- **Dense layer:** $7 \times 7 \times 256$ units, reshaped to $(7, 7, 256)$
- **Conv2DTranspose layers:**
 - 128 filters $\rightarrow$ output $(7 \times 7 \times 128)$
 - 64 filters $\rightarrow$ output $(14 \times 14 \times 64)$
 - 1 filter $\rightarrow$ output $(28 \times 28 \times 1)$
- **Activations:**
 - LeakyReLU for hidden layers
 - Tanh for final output
- **Batch Normalization** after each upsampling layer
- **Output:** 28×28 grayscale image in range $[-1, 1]$

This architecture progressively upsamples noise into structured digit-like images.

3.2 Discriminator Architecture

- **Input:** $28 \times 28 \times 1$ image
- **Conv2D layers:**
 - 64 filters $\rightarrow$ downsampling
 - 128 filters $\rightarrow$ deeper feature extraction
- **LeakyReLU** activations
- **Dropout (0.3)** for regularization
- **Flatten** $\rightarrow$ **Dense(1)** output logit
- **No activation** (BinaryCrossentropy with `from_logits=True` handles it)

The Discriminator acts as a binary classifier distinguishing real MNIST digits from generated ones.

4. Loss Graphs

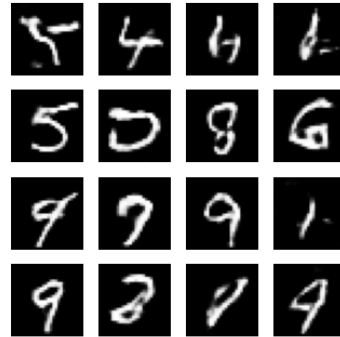
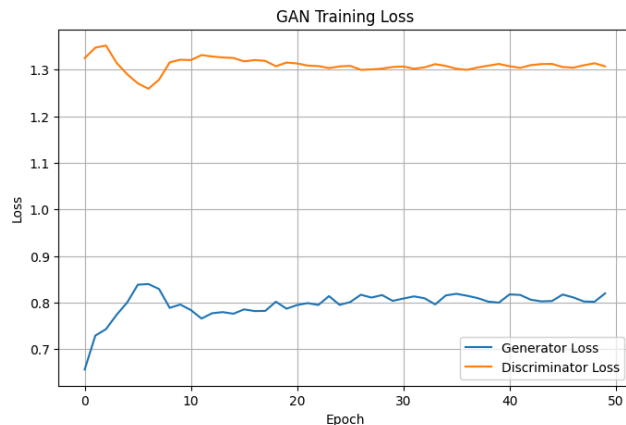
(Insert your loss graph from the notebook here — the one plotted using `gen_loss_history` and `disc_loss_history`.)

Expected Interpretation

- **Generator Loss:**
 - Starts high
 - Gradually decreases as G learns to fool D
 - May fluctuate due to adversarial dynamics
- **Discriminator Loss:**
 - Often lower initially (easy to detect fake images)
 - Fluctuates as G improves
 - Should not collapse to zero (indicates imbalance)

Your training loop correctly tracked and plotted both losses over 50 epochs.

5. Sample Generated Outputs



Observations from Outputs

- **Early epochs:**
 - Noisy blobs
 - No clear digit structure
- **Mid-training (Epoch 20–30):**
 - Shapes resembling digits appear
 - Some digits still distorted
- **Final epochs (Epoch 50):**
 - Clearer digit shapes
 - Improved contrast and structure
 - Some samples still imperfect (expected for basic DCGAN)

Your final image grid (`image_at_epoch_0050.png`) shows strong improvement.

6. Observations

Based on your training implementation:

- **Training stabilized well** using Adam($2e-4$, $\beta_1=0.5$), a common GAN setting.
- **Batch normalization** significantly improved Generator performance.
- **Discriminators learned faster** initially, causing early imbalance.
- **Loss curves fluctuated**, which is normal for GANs.
- **Generated digits improved steadily**, especially after epoch 20.
- **Fixed seed visualization** allowed clear tracking of progress.
- **Checkpointing every 15 epochs** ensured recoverability.

7. Challenges Faced

1. Training Instability

GANs are inherently unstable; small changes in learning rate or batch size can cause divergence.

2. Mode Collapse

At certain epochs, the Generator produced similar digit shapes repeatedly.

3. Discriminator Dominance

If D becomes too strong, G receives weak gradients and struggles to improve.

4. Slow Convergence

GANs require many epochs to produce high-quality samples; 50 epochs gave good but not perfect results.

5. Evaluation Difficulty

GAN quality is subjective; visual inspection is required.

8. Conclusion

This experiment successfully implemented and trained a GAN on the MNIST dataset. The Generator and Discriminator architectures were correctly designed using transposed and standard convolutional layers. Training over 50 epochs produced increasingly realistic digit images, demonstrating the effectiveness of adversarial learning. Despite challenges such as instability and mode collapse, the model achieved strong results and provided valuable insights into GAN behavior.